PUCRS ESCOLA POLITÉCNICA

Fundamentos de Processamento Paralelo e Distribuído (98713-04) — 2026/2 — Prof. Marcelo Veiga Neves

Trabalho T1: Problemas Clássicos de Concorrência

Jantar dos Filósofos e Produtor/Consumidor em Go

Grupo B: Leonardo Noronha Dornelles, Nícolas Michels, Pedro Fialho e Raul Antonio da Silveira
Porto Alegre, outubro de 2026

1 Introdução

Este relatório descreve nossas soluções em Go para o Jantar dos Filósofos e para o Produtor/Consumidor com buffer limitado. Para cada problema explicamos as decisões de projeto, analisamos onde pode haver deadlock e starvation e discutimos os dados de execução coletados. Os programas ficam em `filosofos/` e `produtor-consumidor/` e compartilham o `go.mod` da raiz. O README traz os comandos de execução, os scripts `validar` (testes com `-race`) e `coletar` (experimentos), e a pasta `resultados/` guarda as saídas brutas usadas nas tabelas. Repositório: `https://github.com/leonardondornelles/T1-FFPD`.

2 Problema 1: O Jantar dos Filósofos

2.1 Solução adotada

Cada garfo é um canal com buffer de tamanho 1 (`chan struct{}`). Pegar o garfo é enviar um valor no canal, o que bloqueia se ele já estiver ocupado, e soltar é receber esse valor de volta; a exclusão mútua vem do próprio canal. O envio fica dentro de um `select` junto com um canal de cancelamento `done`, para que um filósofo bloqueado possa desistir se a mesa for cancelada. O programa recebe o número de filósofos (`-n`), o número de refeições (`-r`) e a estratégia (`-strategy`). Os tempos de pensar e de comer são sorteados entre 5 e 15 ms, com um gerador por filósofo: com tempos fixos, todos andariam sincronizados e os dados de fairness não diriam muito.

- **deadlock** (versão base): todos pegam primeiro o garfo da esquerda. Uma barreira faz cada filósofo esperar até que *todos* estejam com o garfo esquerdo antes de tentar o direito. Assim o pior entrelaçamento é forçado e a mesa trava sempre na primeira rodada; uma simples pausa tornaria esse cenário apenas provável.
- **hierarchy** (hierarquia de recursos): todos pegam primeiro o garfo de menor índice. Na prática só o filósofo $N-1$ muda de ordem, porque seus garfos são o $N-1$ e o 0.
- **conductor** (limite de lugares): um canal com capacidade $N-1$ funciona como semáforo de contagem, e no máximo $N-1$ filósofos disputam garfos ao mesmo tempo.

2.2 Análise de deadlock e starvation

Pelas condições de Coffman, o deadlock só acontece se as quatro valerem ao mesmo tempo: há exclusão mútua (um garfo serve a um filósofo por vez), posse e espera (o filósofo segura um garfo enquanto espera o outro), não preempção (ninguém tira o garfo de outro) e pode haver espera circular (cada um espera o garfo que está com o vizinho).

Na versão **deadlock** as quatro condições acontecem e nenhum filósofo avança. Uma goroutine vigia (watchdog) acompanha um contador atômico de refeições; se ele não muda por 2 s, o watchdog fecha `done`, todos os filósofos desistem e o programa informa o deadlock. Ele não impede o deadlock, só o detecta e encerra a mesa. Nessa versão a starvation é total, porque ninguém volta a comer.

A **hierarchy** elimina a espera circular: se todos pegam os garfos em ordem crescente de índice, quem segura o garfo de maior índice nunca espera por um de índice menor, então não há como fechar um ciclo. A **conductor** também elimina a espera circular, mas por contagem: o ciclo precisaria dos N filósofos segurando um garfo cada, e só $N-1$ estão na mesa; com $N-1$ filósofos e N garfos, pelo

menos um consegue os dois garfos. A posse e espera continua existindo na conductor, o que aparece nos dados a seguir.

Nenhuma das duas estratégias garante ausência de starvation: a especificação do Go não promete ordem justa no escalonamento das goroutines nem no desbloqueio dos canais. Nos testes, não observamos starvation: todos os filósofos completaram as R refeições em todas as execuções.

2.3 Análise de fairness

A Tabela 1 mostra a espera pelos garfos com $N = 5$ e $R = 200$, em médias de dez execuções da hierarchy e onze da conductor (`go run ./filosofos -n 5 -r 200 -strategy …`), sem `-race`, que distorceria os tempos, em Go 1.27.0 (windows/amd64), AMD Ryzen 5 5600G. As saídas completas estão em `resultados/`. Como cada filósofo faz exatamente R rodadas, todos fizeram 200 refeições; por isso a fairness aparece no tempo de espera, não na contagem de refeições.

Tabela 1: Espera pelos garfos por filósofo ($N = 5$, $R = 200$, todos com 200 refeições). "Média" é a média das execuções; "Máxima" é a maior espera observada.

	hierarchy		**conductor**	
Filósofo	Média (ms)	Máxima (ms)	Média (ms)	Máxima (ms)
0	8,67	50,9	18,96	54,6
1	6,90	31,3	19,00	51,7
2	6,17	29,3	19,04	51,4
3	5,59	23,8	19,10	54,0
4	8,37	47,6	18,98	51,9
Média geral	7,14		19,02	
Média geral: faixa entre execuções	7,0–7,3		11,1–26,9	
Diferença maior/menor	3,09		0,15	

Os dados mostram uma troca entre velocidade e justiça. A **hierarchy** espera bem menos (7,14 ms contra 19,02 ms), mas de forma desigual: os filósofos 0 e 4 esperam mais que os outros em todas as execuções, e a espera máxima do filósofo 0 (50,9 ms) é mais que o dobro da do filósofo 3 (23,8 ms). A causa é a própria ordenação: os filósofos 0 e 4 disputam o garfo 0 como primeiro garfo. Na **conductor** a diferença entre o filósofo que mais espera e o que menos espera cai para 0,15 ms.

A espera maior da conductor tem duas explicações. A medição inclui o tempo esperando um lugar na mesa. Além disso, como a posse e espera continua, os filósofos sentados podem formar uma fila em que cada um segura o garfo esquerdo esperando o do vizinho, e só o último da fila come; na hierarchy os filósofos 0 e 4 disputam o garfo 0 de mãos vazias, o que corta essas filas. A espera média geral da conductor também variou muito entre execuções (de 11,1 a 26,9 ms), enquanto a da hierarchy ficou entre 7,0 e 7,3 ms. Nossa interpretação, não verificada diretamente, é que as filas formadas nas primeiras rodadas tendem a se manter. Ainda assim, dentro de cada execução da conductor a diferença entre filósofos ficou abaixo de 0,5 ms. Nos dados coletados, a conductor foi mais justa e a hierarchy mais rápida.

2.4 Encerramento ordenado

Cada filósofo é uma goroutine que chama `wg.Done()` ao terminar suas R rodadas ou ao ser cancelado. Toda espera por recurso (garfo, lugar na mesa e barreira) é um `select` que também escuta `done`, então nenhuma espera é infinita. A goroutine principal espera os filósofos (`wg.Wait()`), fecha o canal `fim` para avisar o watchdog e recebe dele, por um canal, a informação de se houve deadlock, o que garante que ele também terminou. Só então imprime a tabela; na versão deadlock, ela mostra as refeições feitas (nenhuma) e avisa que as esperas em andamento foram interrompidas pelo cancelamento. Parâmetros inválidos são recusados antes de criar qualquer goroutine.

3 Problema 2: Produtor/Consumidor com Buffer Limitado

3.1 Solução adotada

As duas versões ficam no mesmo programa, escolhidas por `-mode channel` ou `-mode semaphore`, e implementam a mesma interface `Buffer` (`Put`, `Get`, `Len` e `Fechar`). Produtores, consumidores, encerramento, verificação e medições são o mesmo código nos dois modos, de modo que a diferença de desempenho vem da implementação do buffer.

Versão com channel. O buffer é um `chan int` de capacidade K, e o bloqueio com buffer cheio ou vazio vem do próprio canal.

Versão com semáforos. O buffer é um vetor circular de tamanho K com índices `in` e `out`, seguindo o algoritmo clássico visto em aula. Os semáforos de contagem `notFull` (começa com K fichas) e `notEmpty` (começa com 0) são canais de fichas com as operações `P` e `V`. O produtor faz `P(notFull)`, trava `mutexIn`, escreve na posição `in`, destrava e faz `V(notEmpty)`; o consumidor faz o caminho inverso com `notEmpty`, `mutexOut` e `notFull`. O `mutexIn` impede que dois produtores escrevam na mesma posição e o `mutexOut` impede que dois consumidores retirem o mesmo item. Um único mutex também atenderia à exigência de exclusão mútua; usar dois foi uma escolha nossa, para que um produtor e um consumidor não precisem esperar um pelo outro. Quem impede que os dois mexam na mesma posição são os semáforos: `notEmpty` só libera posições já escritas e `notFull` só libera posições já lidas. Pelo modelo de memória do Go, o envio em um canal acontece antes do recebimento correspondente, o que garante que o consumidor enxerga o item escrito pelo produtor.

Verificação. Cada produtor gera identificadores únicos, e um vetor conta quantas vezes cada identificador foi consumido. Ao final, o programa confere que o total produzido é igual ao consumido, que *cada* identificador foi consumido exatamente uma vez e que o buffer terminou vazio, e termina com erro se algo não bater.

Consumo com timeout. O consumidor 0 lê com `select` e `time.After`. Quando o tempo acaba, ele registra o timeout no log e volta a esperar. Ele não encerra, porque um timeout só indica que a produção está lenta, não que acabou; se saísse, os produtores poderiam ficar presos com o buffer cheio. Para fins de comparação, `-timeout 0` desliga esse comportamento.

3.2 Análise de deadlock e starvation

Na versão com semáforos, a ficha é sempre obtida antes do mutex, e quem segura um mutex não espera por mais nada: só escreve ou lê uma posição e o libera. Um produtor pode segurar uma ficha de `notFull` enquanto espera o `mutexIn`, então há posse e espera; mas, como a ordem de aquisição é sempre a mesma, nenhum ciclo pode se formar, e a condição eliminada é a espera circular. Com o buffer cheio os consumidores conseguem esvaziá-lo, e com ele vazio os produtores conseguem enchê--lo. Na versão com channel, o canal é fechado depois que todos os produtores terminam, e nenhum consumidor fica esperando para sempre.

Não ter deadlock não garante divisão justa entre consumidores, e nem o Go nem o algoritmo garantem espera limitada. Nos benchmarks, todos os consumidores apresentaram contagem média positiva, embora o consumidor com timeout receba menos (Seção 3.3); esses resultados não garantem ausência de starvation em outras execuções.

3.3 Análise do efeito do buffer

As Tabelas 2 e 3 foram geradas com `go run ./produtor-consumidor -bench -p 4 -c 4 -n 125000`, com e sem `-timeout 0` ($P = 4$, $C = 4$, 500.000 itens, cinco repetições por combinação), no mesmo ambiente da seção anterior; as saídas brutas estão em `resultados/`. O throughput é o total de itens dividido pelo tempo do início até o último consumidor terminar. O tempo inclui o encerramento e a contabilização atômica por identificador, mas exclui a conferência final dos resultados. O desvio padrão do throughput entre as repetições ficou entre 1,6% e 6,1%. A ocupação média vem de uma goroutine que lê o tamanho do buffer enquanto os produtores trabalham, num intervalo pedido de 100 μs; o sistema pode atrasar essas leituras, então é uma amostragem aproximada, suficiente para comparar os valores de K.

Efeito de K. Com o timeout ligado, o throughput cresceu cerca de 2 vezes de $K = 1$ para

Tabela 2: Efeito de K com o consumidor 0 usando timeout de 10 ms ($P = 4$, $C = 4$, 500.000 itens, média de 5 execuções; desvio padrão do throughput entre parênteses).

Versão	K	Throughput (itens/s)	Ocupação	Itens por consumidor (mil) C0 / C1 / C2 / C3
channel	1	1.173.914 (6,1%)	0,52	78,5 / 139,9 / 141,0 / 140,7
channel	10	1.383.919 (1,6%)	5,25	70,7 / 143,3 / 142,0 / 144,1
channel	100	2.327.898 (6,0%)	54,19	59,6 / 147,1 / 145,5 / 147,8
semaphore	1	530.458 (1,9%)	0,43	123,6 / 125,5 / 125,4 / 125,5
semaphore	10	714.442 (3,1%)	5,55	103,4 / 131,2 / 133,2 / 132,3
semaphore	100	1.114.997 (3,2%)	54,46	103,2 / 132,0 / 131,1 / 133,6

Tabela 3: Mesma medição com o timeout desligado (`-timeout 0`): todos os consumidores usam a leitura bloqueante comum.

Versão	K	Throughput (itens/s)	Ocupação	Itens por consumidor (mil) C0 / C1 / C2 / C3
channel	1	1.163.639 (4,2%)	0,51	124,6 / 125,1 / 124,6 / 125,6
channel	10	1.872.534 (3,6%)	4,98	126,6 / 124,0 / 125,5 / 123,9
channel	100	3.198.079 (1,7%)	49,83	121,2 / 133,2 / 120,7 / 124,9
semaphore	1	731.432 (4,9%)	0,47	125,0 / 125,0 / 125,0 / 125,0
semaphore	10	789.046 (2,6%)	5,64	124,7 / 124,7 / 125,4 / 125,2
semaphore	100	1.205.628 (3,6%)	47,10	123,8 / 124,2 / 125,2 / 126,8

$K = 100$ nas duas versões (2,0 no channel e 2,1 nos semáforos); sem o timeout, 2,7 e 1,6 vezes. Com $K = 1$ o produtor já pode deixar um item e seguir sem esperar um consumidor, mas só um: se o item ainda não foi retirado, o próximo produtor bloqueia. A ocupação média perto de 0,5 indica que o único lugar ficou ocupado cerca de metade do tempo; com uma só posição, há pouca folga para absorver diferenças de ritmo. Com $K = 100$ a ocupação média fica entre 47 e 54, e o buffer oferece maior capacidade para absorver diferenças de ritmo entre produtores e consumidores, o que é o desacoplamento que um K maior proporciona. O ganho não é proporcional: multiplicar K por 10 aumentou o throughput de 1,1 a 1,7 vez. Uma explicação plausível é que, a partir de certo ponto, o custo de sincronizar e processar cada item pesa mais que a falta de espaço.

Channel ou semáforos. A versão com channel foi de 1,6 a 2,7 vezes mais rápida. Somando produção e consumo, cada item custa no channel um envio e um recebimento, operações do runtime que podem entregar o item direto a um consumidor que já está esperando. Na versão com semáforos, o mesmo item custa quatro operações de semáforo (`P` e `V` de cada lado) e duas aquisições de mutex. Sem o timeout, a diferença cresceu com K (de 1,6 para 2,7 vezes), o que é coerente com esse custo fixo por item pesar mais à medida que K cresce; com o timeout, ficou em torno de 2 vezes em todos os K. A vantagem da versão com semáforos é mostrar cada passo do algoritmo clássico, não a velocidade.

Distribuição entre consumidores. Com o timeout, o consumidor 0 recebeu de 48% a 63% da média no channel e de 83% a 99% nos semáforos. Sem o timeout (Tabela 3), recebeu de 97% a 101% da média em todas as combinações. A comparação mostra que a diferença vem do consumo com timeout; o mecanismo mais provável é a criação de um timer a cada volta do laço, mesmo quando há item disponível. Desligar o timeout também aumentou o throughput total em até 38% (exceto no channel com $K = 1$, em que ficou igual), o que sugere que o consumidor mais lento limita o conjunto. Nos benchmarks, todos os consumidores apresentaram contagem média positiva; esses resultados não garantem ausência de starvation em outras execuções.

3.4 Encerramento ordenado

Na versão com channel, depois que o `WaitGroup` dos produtores termina, o canal é fechado. Os consumidores continuam retirando itens até o canal esvaziar, quando a leitura devolve `ok == false` e eles saem; nenhum item se perde e ninguém envia em canal fechado. Na versão com semáforos, depois dos produtores, inserimos uma sentinela por consumidor no próprio buffer, pelo mesmo caminho dos itens. Como o buffer é FIFO e a produção já acabou, nenhum item fica atrás das sentinelas, e cada consumidor sai ao retirar a sua. Isso funciona mesmo com K menor que o número de consumidores,

porque sempre sobra pelo menos um consumidor ativo para abrir espaço. Uma alternativa seria fechar o canal de `notEmpty`, mas isso exigiria adaptar `P` e a leitura, já que receber de um canal fechado retorna imediatamente; as sentinelas mantêm o semáforo apenas com `P` e `V`.

4 Validação

O script `validar` (versões `.ps1` e `.sh`) roda `go vet` e 118 casos com o detector de corridas: as três estratégias dos filósofos com N igual a 2, 5 e 20; o produtor/consumidor nos dois modos com K igual a 1, 2, 10 e 100, várias combinações de P e C (inclusive $K < C$) e 0, 1 e 500 itens por produtor; timeout com produtor lento; o modo sem timeout; e parâmetros inválidos. Todos passaram sem aviso de data race. Para conferir que o script detecta erros, removemos o `mutexOut` de uma cópia do código, e ele acusou falhas. Em uma cópia do programa dos filósofos que imprime `runtime.NumGoroutine()` no final, nenhuma goroutine sobrou em nenhuma estratégia. O detector de corridas só cobre as execuções que de fato ocorreram, então isso não prova ausência de data races. No Windows, o `-race` exige um compilador C (usamos o MinGW).

5 Uso de Ferramentas de IA

Usamos assistentes de IA em várias etapas, e todos os integrantes revisaram o resultado e conseguem explicar o código e as análises. **Organização:** um assistente de IA ajudou a montar a divisão de tarefas e o roteiro inicial de cada parte. **Problema 1:** o Claude foi usado na implementação dos filósofos, na discussão das estratégias e das condições de Coffman e na depuração, incluindo o watchdog e o sorteio dos tempos, e como apoio na redação da análise. **Problema 2:** a versão com channel e o consumo com timeout foram feitos inicialmente com ajuda do Codex; a versão com semáforos, as medições e os experimentos, com ajuda do Claude, no projeto da solução, na geração do código, na depuração, na análise dos dados e na redação. **Revisão final:** o Claude foi usado para conferir o código com o enunciado, simplificar e reorganizar os dois programas, aplicar as correções de uma revisão técnica (barreira no deadlock, verificação por identificador, scripts de validação e coleta), rodar a validação com `-race` e atualizar este relatório. Os dados das tabelas foram coletados pelo grupo.

6 Referências

1. DIJKSTRA, E. W. Hierarchical ordering of sequential processes. *Acta Informatica*, v. 1, n. 2, p. 115–138, 1971.
2. COFFMAN, E. G.; ELPHICK, M. J.; SHOSHANI, A. System deadlocks. *ACM Computing Surveys*, v. 3, n. 2, p. 67–78, 1971.
3. SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. 9. ed. Rio de Janeiro: LTC, 2015.
4. TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson, 2016.
5. THE GO AUTHORS. *The Go Programming Language Specification*. `https://go.dev/ref/spec`.
6. THE GO AUTHORS. *The Go Memory Model*. `https://go.dev/ref/mem`.
7. THE GO AUTHORS. *Data Race Detector*. `https://go.dev/doc/articles/race_detector`.
8. THE GO AUTHORS. Documentação dos pacotes `sync`, `sync/atomic`, `time`, `math/rand` e `flag`. `https://pkg.go.dev/std`.
9. Material da disciplina de Fundamentos de Processamento Paralelo e Distribuído (PUCRS, 2026/2).